

EECS 498 · AASE · FALL 2026

Spec-Driven Coding

LECTURE 04 · SEP 10, 2026

WHERE WE ARE

Today: the Prompt lever

- L03 gave you three levers and one instruction: be concrete
- The failure that dominates is still missing context
- A well-structured prompt changes how you manage context too

Concrete was the instruction. Today it becomes a discipline.

What you leave with today

- The vocabulary of a precise prompt
- The structure that scales it past one request
- When a spec is worth writing, and when it is not
- Architect Mode, which runs a spec in one pass

Everything here is aimed at Lab01 and the build that follows it.

IDKs

INFORMATION DENSE KEYWORDS

Vague

Add another way
to output the data.

Model guesses every detail.

IDK-rich

```
CREATE output_format.py with  
    format_as_html(...) -> str  
UPDATE main.py: ADD html  
    to --output-format choices
```

Nothing left to guess. Usually right the first time.

The vocabulary



CREATE · ADD

CREATE makes what does not exist. ADD appends to what does.



UPDATE · REPLACE

UPDATE modifies in place. REPLACE swaps the implementation.



DELETE · REMOVE

DELETE takes out the whole thing. REMOVE takes out one line or argument.



MOVE · MIRROR

MOVE relocates code. MIRROR copies a pattern into a new file.

The pair is the point. ADD $\neq$ CREATE.

[ACTION] [LOCATION]: [DETAIL]

Pattern in practice

```
CREATE main.py with  
    analyze_transcript(filepath: str) -> TranscriptAnalysis
```

```
UPDATE output_format.py: ADD  
    format_as_yaml() after format_as_json()
```

```
DELETE main.py: REMOVE  
    the unused print_debug() function
```


Narrow the sampling distribution

- Vague prompt → wide distribution → model picks training-prior
- Precise prompt → narrow distribution → model picks what you said
- IDKs work because of *mechanics*, not English fluency

Prompt Levels

HIGH • MID • LOW

Where to operate



High

"Make it better." Natural, risky.



Mid · sweet spot

"UPDATE chart.py:
word_count_bar_chart() — top
quartile green, sort desc."



Low

"Line 47, replace X with Y."
Verbose, typist.

The sweet spot shifts with the model. Frontier tolerates higher; a 4B needs more.

Plan = Prompt

THE PLANNING BOTTLENECK

The bottleneck has shifted

- **2024:** can the AI understand what I want?
- **2026:** can *I* specify what I want?
- Quality of output bounded by quality of plan
- AI = executor. You = architect.
- A human contractor asks questions when the spec is vague. An LLM guesses.

The spec **IS** the prompt.

When to use a spec



Yes

3+ files coordinated · interdependent steps ·
reproducibility · build-from-scratch · archive



No

Single bug fix · one-fn refactor · "while you're in here, also
X"

If you would write a design doc for a human contractor, write a spec.

taskr

The task manager you have been living in. `task.py` is the record, `store.py` is the JSON on disk, `cli.py` is the commands. So far it can only `add` and `list`.

- `taskr done 3` — mark a task complete
- `taskr rm 3` — delete a task
- `taskr list --tag work` — filter by tag
- `taskr add "... " --due fri` — a due date, shown in the listing
- `taskr list --json` — machine-readable output

Pick one, or bring your own. Either way, name the file and the function.

CHAT BREAK • 5 MIN

Take a change you'd make to `taskr`.
Write it as ACTION LOCATION: DETAIL,
then the **highest level that still
lands first try.**

PollEv

<https://PollEv.com/marcusdarden365>

Spec Anatomy

FIVE SECTIONS

Spec document

1. **High-Level Objective** — "what" and "why", generally stated
2. **Mid-Level Objectives** — 3-5 measurable outcomes
3. **Implementation Notes** — deps, standards, constraints
4. **Context** — beginning + ending files
5. **Ordered Low-Level Tasks** — IDK-rich, dependency-ordered

Objectives and notes



High-level

One sentence of why. "Add word frequency visualization and structured file output to the transcript analytics CLI."



Mid-level

Three to five outcomes, each independently verifiable. Chart functions, output formatters, two new CLI flags.



Notes

Use matplotlib, it's already a dependency. Follow the patterns in the project. Briefly comment every new function.

Objectives say what done looks like. Notes say what the model must not guess.

Context: beginning + ending

```
### Beginning context
- src/main.py
- src/data_types.py (read-only)
- pyproject.toml (read-only)

### Ending context
- src/main.py (updated)
- src/chart.py (new file)
- src/output_format.py (new file)
```

Eliminates ambiguity. Model knows exactly what's in scope.

Compounding sequence

1. CREATE `output_format.py` with
`format_as_text/json/md/yaml`
2. CREATE `chart.py` with
`create_bar/pie/line_chart` –
`colors: top green, bottom red`
3. UPDATE `main.py`: ADD `--chart`,
`--output-format`, call new fns

Task 3 references Task 1's outputs because Task 1 ran first.

Architect Mode

SPEC IN ACTION

Architect Mode in Aider

```
aider --architect \  
      --model ollama_chat/qwen3.5:9b \  
      --editor-model ollama_chat/qwen3.5:9b
```

- **Architect** — drafts the plan in prose
- **Editor** — turns the plan into edits
- `auto-accept-architect: false` — the plan stops. Read it.

Same model in both seats. The demo's config sets all three, so on screen you will see just `aider src/main.py src/arg_parse.py`.

Watch it run

1. `spec.md` on screen. Walk the five sections.
2. `aider src/main.py src/arg_parse.py` — Architect Mode, no flags.
3. Paste the spec. The architect writes a plan in prose.
4. **Read the plan out loud, then `y`**. Nothing is on disk yet.
5. Four files: `output_format.py`, `chart.py` new; `arg_parse.py`, `main.py` updated.
6. `uv run main transcript.txt --chart bar --output-format json`

Common Mistakes

BEFORE YOU WRITE YOUR OWN

What graders see

- **Vague MLOs** — "make it better" sneaks in
- **Missing context section** — model touches files it shouldn't
- **Out-of-order tasks** — Task 1 references Task 3
- **Code in tasks** — implementation, not instructions

Specs are communication artifacts. Treat them as such.

What to take away

- **IDKs** name the action and the location, which narrows the sampling distribution
- **Mid-level** is the sweet spot, and the sweet spot moves with the model
- **Plan = Prompt.** The spec is the instruction, not a document about it
- **Ordered tasks** compound, so later steps lean on earlier ones
- **Architect Mode** splits planning from editing. One job per call
- **Specs earn their cost** on three or more coordinated files

Three questions

1. How is a spec different from one very long prompt?
2. Would you write a spec for a bug fix? Where does it become overkill?
3. Why put "comment every function" in the spec instead of hoping for it?

THIS WEEK

Lab01 + the build

- **Tomorrow (Fri Sep 11)** — Aider setup gate + all sixteen lessons
- **Mon Sep 14 / Tue Sep 15** — Lab01, the pivot lab
- **Fri Sep 25** — the build is due (end of week 4)
- **Not yet** — `aider-practice` finishes this week; the build starts after Lab01

The seven features

A terminal coding assistant: conversation, files in context, model-written edits applied behind a diff.

Required behavior

- | | |
|-----------|--|
| F1 | Conversation, budget handling, streamed replies |
| F2 | Add, drop, list and clear context |
| F3 | Current file contents, rendered consistently |
| F4 | An edit-format prompt you have tested |
| F5 | Parsing with useful errors |
| F6 | Exact edits, complete preview, explicit approval |
| F7 | One commit per accepted edit set, guarded undo |

BOTH ARE GRADED

What this feeds



Your build's spec doc

Seven features on a starter is the "three or more files, from scratch" case. The spec is a required deliverable.



Feature F4, the edit-block prompt

A spec written for one small model. Leave it vague and your parser rejects the output, so nothing lands on disk.

Everything today is about to become code you hand in.

Modern Agentic CLIs

- The same spec you write today, handed to Claude Code and to Codex
- A demo from the podium. No account, nobody is asked to buy anything
- More autonomy, different behavior, and the Big Three still applies

The mature version of the thing you are about to build. Tuesday.

Questions?